Attention

How Transformers Decide What to Focus On

---

1. What Problem Attention Solves

Older sequence models (RNNs, LSTMs) compress an entire input sequence into a single fixed-size hidden vector before producing output, creating a bottleneck for long sequences. Attention lets a model look back at **all** input positions at every output step and decide, dynamically, which ones matter most right now.

2. The Core Idea: Query, Key, Value

Every attention mechanism reduces to comparing a **query** (what am I looking for) against a set of **keys** (what each position offers), producing weights that are used to combine the corresponding **values**.

- **Query (Q)** - representation of the current position asking a question.
- **Key (K)** - representation of each position being searched, used for matching.
- **Value (V)** - the actual content that gets aggregated once relevance is known.

3. Scaled Dot-Product Attention

The dominant formulation, introduced in the Transformer architecture, computes attention as:

```
Attention(Q, K, V) = softmax( Q K^T / sqrt(d_k) ) V
```

Steps:

- Compute similarity scores between every query and every key via a dot product.
- Scale by 1/sqrt(d_k) to keep gradients stable as the key dimension d_k grows.
- Apply softmax across the key dimension so weights sum to 1 for each query.
- Use those weights to take a weighted sum of the value vectors.

Note: Scaling matters because large dot products push softmax into regions with vanishing gradients.

4. Self-Attention vs Cross-Attention

- **Self-attention**: Q, K, and V all come from the same sequence - lets each token attend to every other token in the same input (e.g., a sentence attending to itself).
- **Cross-attention**: Q comes from one sequence, K and V come from another (e.g., a decoder attending to an encoder's output, as in translation or retrieval-augmented generation).

5. Multi-Head Attention

Rather than computing a single attention distribution, the model runs several attention 'heads' in parallel, each with its own learned Q, K, V projections. This lets different heads specialize in different types of relationships (syntax, position, coreference, etc.).

```
MultiHead(Q,K,V) = Concat(head_1, ..., head_h) W_O head_i = Attention(Q W_i^Q, K W_i^K, V W_i^V)
```

6. Positional Information

Attention has no inherent notion of order - it treats the input as a set. Positional encodings (sinusoidal, learned, or rotary embeddings such as RoPE) are added or fused in so the model can distinguish 'token 3 before token 7' from an unordered bag of tokens.

7. Masking

- **Causal mask**: in decoders/generation, blocks a position from attending to future tokens, enforcing left-to-right generation.
- **Padding mask**: prevents attention to padding tokens added to equalize sequence lengths in a batch.

8. Complexity and Efficiency

Standard self-attention costs O(n^2) in time and memory for sequence length n, since every token attends to every other token. This motivates sparse, linear, and windowed attention variants (e.g., Longformer, FlashAttention, sliding-window attention) for long-context models.

9. Why Attention Matters

- Enables the Transformer architecture, the backbone of modern LLMs.
- Captures long-range dependencies without the sequential bottleneck of RNNs.
- Parallelizes across the sequence dimension, making training on GPUs/TPUs far faster.
- Provides some interpretability - attention weights show what a model 'looked at'.